

React-da Server State İdarəolunması

useEffect əzabından TanStack Query ilə təmiz koda doğru — asinxron prosesləri əllə idarə etməkdən yorulan tərtibatçılar üçün praktiki bələdçi.

REACT

TYPESCRIPT

TANSTACK QUERY

Server State Nədir?

Aləti seçməzdən əvvəl, tam olaraq nə ilə işlədiyimizi anlayaq. React tətbiqlərində prinsipial olaraq iki fərqli vəziyyət (state) növü mövcuddur:

Client State

Yalnız brauzerdə yaşayır, sinxronudur və tək bir istifadəçiyə məxsusdur.

- Mövzu (qaranlıq / işıqlı)
- Açıq modallar və dropdown-lar
- Lokal formaların dəyərləri
- Aktiv tab və ya slider vəziyyəti

Server State

Bəkkenddə saxlanılır, asinxronudur və istənilən an dəyişə bilər.

- İstifadəçi, sifariş və məhsul siyahıları
- Profil və hesab tənzimləmələri
- Digər istifadəçilərin dəyişdiyi məlumatlar
- Fetch / Axios tələb edən hər şey

Qayda: Server State xüsusi yanaşma — *keşləmə, sinxronizasiya və xataların idarə olunmasını tələb edir*. Standart `useState` təkbaşına bunun öhdəsindən gələ bilmir.

Problem: useEffect + fetch əzabı təfərrüatları ilə

Klassik yanaşma bizi hər bir sorğu üçün əllə üç müstəqil state hissəsini idarə etməyə məcbur edir:

```
function UserProfile() {
  const [data, setData] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
    fetch('/api/users/1')
      .then(res => res.json())
      .then(setData)
      .catch(setError)
      .finally(() => setIsLoading(false));
  }, []);
```

```
if (isLoading) return <Spinner />;  
if (error) return <ErrorBox error={error} />;  
  
return <div>{data?.name}</div>;  
}
```

Boilerplate

Hər sorğu üçün üç state — kod eksponensial olaraq böyüyür və idarəolunmaz həddə çatır.

Keş yoxdur

Hər render zamanı yeni sorğu atılır, hətta məlumatlar heç dəyişməsə belə təkrar yüklənir.

Race Conditions

Səhifələrin sürətli dəyişməsi zamanı asinxron cavablar səhv ardıcılıqla gəlib interfeysi korlaya bilər.

Mürəkkəblilik

Eyni sorğuların təkrarlanmaması (deduplication), uğursuz olduqda yenidən sınıanması — hamısı sizin boynunuzdadır.

Həll: TanStack Query

Fəlsəfə sadədir: state-i imperativ idarə etmək əvəzinə, əldə etmək istədiyiniz məlumatı deklarativ şəkildə təsvir edirsiniz. Kitabxana bütün rutin işləri öz üzərinə götürür.

Daxili Keş Sistemi

Məlumatlar yaddaşda saxlanılır, komponentlər və naviqasiya arasında təkrar istifadə olunur.

Arxa Plan Sinxronizasiyası

Məlumatlar arxa planda yenilənir, bu zaman interfeys donmur və çevik qalır.

Sorğuların Dublikatının Qarşısı

Eyni vaxtda atılan eyni sorğular birləşdirilir — şəbəkə trafiki boş yerə yüklənmir.

Xətalərin Avtomatik Emalı

Hazır `isError` və `error` flag-ləri + şəbəkə kəsildikdə avtomatik təkrar cəhdlər (retries).

useQuery: Xykun Anatomiyası

```
const { data, isLoading, isError, error, isFetching } = useQuery({
  queryKey: ['users', userId],
  queryFn: () => fetch(`/api/users/${userId}`).then(res => res.json())
});
```

Geri Qayıdan Flag-lər:

- **data** — əldə edilmiş məlumat (ilk uğurlu sorğudan sonra mövcud olur).
- **isLoading** — yalnız ilk sorğu zamanı, yaddaşda hələ heç bir keşlənmiş məlumat olmadıqda `true` olur.
- **isFetching** — istənilən sorğu zamanı, o cümlədən arxa planda yenilənmə (refetch) zamanı `true` olur.
- **isError / error** — xətanın statusu və xəta obyektinin özü.

`queryKey` — keşin unikal identifikatorudur. Massiv formatı onu parametrləşdirməyə imkan verir: `['users', 42]` və `['users', 99]` tamamilə fərqli keş qeydləridir.

Kritik fərq: `isLoading` yalnız ilk işəsalınmaya aiddir. `isFetching` isə arxa plandakı yenilənmələr də daxil olmaqla hər bir sorğunu əhatə edir. Tab-lar arası keçid zamanı spinner göstərmək üçün `isFetching` flag-indən istifadə edin.

useMutation: Məlumatların Dəyişdirilməsi

POST / PUT / DELETE sorğuları üçün ayrıca hook istifadə olunur. O, avtomatik icra olunmur: sorğunun göndərilmə anını `mutate()` funksiyası vasitəsilə özünüz idarə edirsiniz.

```
const mutation = useMutation({
  mutationFn: (newUser: CreateUserDto) =>
    fetch('/api/users', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(newUser)
    }),
  onSuccess: (data) => {
    queryClient.invalidateQueries({ queryKey: ['users'] });
  },
  onError: (error) => {
    toast.error('Xəta baş verdi, icra olunmadı');
  }
});

// Komponent daxilində çağırış:
mutation.mutate({ name: 'Alice', age: 28 });
```

Niyə ayrı bir hook?

- Mutasiyalar keşlənmir — onlar məlumatı oxumur, serverdəki məlumatı dəyişdirirlər.
- `onSuccess` — əlaqəli keşləri qüvvədən salmaq (`invalidate`) üçün ideal yerdir.
- `onError` — formadakı xətalara mərkəzləşdirilmiş şəkildə idarə etmək üçün əlverişlidir.
- `isPending`, `isSuccess` flag-ləri düymələrin vizual vəziyyətlərini (UX) idarə etməyə kömək edir.

Pattern: Formanı göndərdiniz → `onSuccess` aktivləşdi → `invalidateQueries` icra olundu → Siyahı avtomatik olaraq yeniləndi.

TanStack Query-nin Ürəyi: `staleTime` və `gcTime`

İki parametr məlumatların keşdəki həyat dövrünü idarə edir. Onların fərqi anlamaq anı reaksiya verən interfeys (Instant UI) və şəbəkə qənaətinin açarıdır.

- **Təzə Məlumatlar (Fresh Data):** Data yüklənib və istifadə olunur. Şəbəkə sorğusu atılmır. Tərəvət müddəti (`staleTime`) aktivdir.
- **Köhnəlmiş Məlumatlar (Stale Data):** `staleTime` müddəti bitib. Köhnə məlumat ekranda qalır, lakin arxa planda yenilənmə (`refetch`) başlayır.

- **Zibil Yığıcı (Garbage Collection):** Komponent unmount olunub. Saxlanma müddəti (`gcTime` / `cacheTime`) geri sayır.
- **Silinib:** Müddət bitdikdən sonra məlumat yaddan tamamilə təmizlənir.

staleTime (default 0): Məlumatların "təzə" hesab edildiyi vaxt. Bu müddət bitənə qədər React Query arxa plan sorğusu atmadan birbaşa keşdən məlumatı qaytarır. Məsələn: `staleTime: 5 * 60 * 1000` (5 dəqiqə sorğusuz rejim).

gcTime / cacheTime (default 5 dəq): Komponentlər tərəfindən istifadəsi dayandırıldıqdan sonra məlumatın yaddan silinməməsi üçün keçən müddət. Naviqasiya zamanı yaddaşa qənaət edir.

Keşin Invalidasiyası: Ağrısız Sinxronizasiya

Problem: Siz useMutation vasitəsilə yeni istifadəçi yaratdınız, lakin ekrandakı siyahı köhnə qalıb.

Həll: queryClient.invalidateQueries() lazımı useQuery-ni avtomatik olaraq yenidən işə salacaq.

```
const queryClient = useQueryClient();
const createMutation = useMutation({
  mutationFn: (user: CreateUserDto) => api.createUser(user),
  onSuccess: () => {
    // ['users'] açarı olan bütün sorğuları köhnəlmiş kimi qeyd edir
    // React Query onları avtomatik olaraq arxa planda yenidən çəkir
    queryClient.invalidateQueries({ queryKey: ['users'] });
    toast.success('İstifadəçi uğurla yaradıldı');
  }
});
```

Heç bir əl ilə setData çağırışlarına, heç bir sinxronizasiya pozuntularına ehtiyac yoxdur — uğurlu mutasiyadan sonra ekrandakı məlumatlar həmişə aktualdır.

useEffect + fetch vs TanStack Query

Kriteriya	useEffect + fetch	TanStack Query
State idarə olunması	Əllə: data, isLoading, error	Avtomatik, quraşdırılmış şəkildə
Keşləmə	Yoxdur — hər render = yeni sorğu	Parametrləri olan daxili ağıllı keş
Race Conditions	Əllə qorunma lazımdır (AbortController)	Avtomatik idarə olunur
Arxa Plan Yenilənməsi	Mürəkkəb əl ilə reallaşdırma	Hazır funksional (staleTime + refetch)
Dublikat Sorğular	Əllə tənzimlənir	Avtomatik qarşısı alınır
Keşin Invalidasiyası	Əllə state-in manipulyasiyası	Tək sətirlə invalidateQueries()
Təkrar Cəhdlər (Retries)	Əllə yazılmalıdır	Hazır gəlir (default olaraq retry: 3)

Yekun: Nəyi harada istifadə etməli?

✓ TanStack Query istifadə edin:

- Serverdən gələn məlumatlar (siyahılar, profillər, sifarişlər)

- Keşləmə və arxa plan yenilənməsi tələb olunduqda
- Serverə məlumat göndərən formalar (POST/PUT/DELETE)
- Eyni məlumatı oxuyan çoxsaylı asılı komponentlər olduqda

✓ **useState / useReducer kifayətdir:**

- Dizayn mövzusu (theme), modallar, dropdown-lar
- Göndərilənə qədər formaların lokal dəyişən dəyərləri
- Sırf UI vəziyyətləri: aktiv tab, sıralama (sorting) kriteriyaları
- Serverlə heç bir əlaqəsi olmayan daxili məlumatlar

Əsas nəticə: TanStack Query — Zustand və ya Redux-un əvəzləyicisi deyil. Bu, server state-i üçün ixtisaslaşdırılmış alətdir. HTTP sorğularının olduğu yerlərdə ondan istifadə edin — kodunuz daha təmiz, etibarlı və sürətli olacaq.